

Лабораторна робота №3

Тема: Дослідження безпечної реалізації та експлуатації децентралізованих додатків

Мета: Отримання навичок роботи із децентралізованими додатками та оцінка безпеки інформації при їх функціонуванні

ФБ-51мн Приходько Юрій

Для виконання роботи обрано платформу Ethereum з локальною тестовою мережею Hardhat. Як децентралізований додаток реалізовано систему Proof of Exploit - реєстр авторства знайдених вразливостей на основі блокчейну.

Використані інструменти - Hardhat 3.7.0, Solidity 0.8.28, Ethers.js 6.13.2, MetaMask, Node.js. Ідея додатку полягає в тому що дослідник безпеки, знайшовши вразливість, хешує її опис локально і записує хеш у блокчейн. Це фіксує факт знання вразливості з точним часом до моменту публічного розкриття. Пізніше будь-хто може верифікувати авторство, надавши оригінальний текст - контракт порівняє хеші.

Смарт-контракт ProofOfExploit

Контракт зберігає для кожного хешу адресу дослідника, мітку часу блоку, рівень критичності та назву цільової системи. Повторна реєстрація одного хешу блокується на рівні контракту.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ProofOfExploit {
    struct Record {
        address researcher;
        uint256 timestamp;
        string severity;
        string target;
    }

    mapping(bytes32 => Record) public records;
    bytes32[] public hashes;

    event Registered(bytes32 indexed hash, address researcher, string
severity, string target, uint256 timestamp);

    function register(bytes32 hash, string calldata severity, string calldata
target) external {
```

```

        require(records[hash].timestamp == 0, "already registered");
        records[hash] = Record(msg.sender, block.timestamp, severity, target);
        hashes.push(hash);
        emit Registered(hash, msg.sender, severity, target, block.timestamp);
    }

    function verify(bytes32 hash) external view returns (address, uint256,
string memory, string memory) {
        Record memory r = records[hash];
        require(r.timestamp != 0, "not found");
        return (r.researcher, r.timestamp, r.severity, r.target);
    }

    function count() external view returns (uint256) {
        return hashes.length;
    }
}

```

Функція register приймає хеш, рівень критичності і назву цілі. Хешування відбувається на стороні клієнта через кесак256 від тексту опису - сам текст у блокчейн не потрапляє, що забезпечує конфіденційність деталей вразливості до офіційного розкриття. Функція verify є view-функцією і не витрачає газ.

```

import { ethers } from "ethers";
import artifact from
"../artifacts/contracts/ProofOfExploit.sol/ProofOfExploit.json" with { type:
"json" };

const provider = new ethers.JsonRpcProvider("http://127.0.0.1:8545");
const signer = await provider.getSigner(0);

const factory = new ethers.ContractFactory(artifact.abi, artifact.bytecode,
signer);
const contract = await factory.deploy();
await contract.waitForDeployment();
console.log("deployed:", await contract.getAddress());

```

```
gratigomarchlinux < 21:41:39
node scripts/deploy.mjs
deployed: 0x5FbDB2315678afecb367f032d93f642f64180aa3

gratigomarchlinux < 21:41:41
Account #12: 0xfabb0ac9d68b0b445fb7357272ff202c5651694a (10000 ETH)
Private Key: 0xa267530f49f828020edf313ee7af6b827f2a8bce2897751d06a843f644967b1
Account #13: 0x1cbd3b2770909d4e10f157cab84c7264073c9ec (10000 ETH)
Private Key: 0x47c99abed3324a2707c28affff1267e45918ec8c3f20b8aa892e8b065d2942dd
Account #14: 0xdf3e18d64bc6a983f673ab319ccae4f1a57c7097 (10000 ETH)
Private Key: 0xc526ee95bf44d8fc405a158bb884d9d1238d99f0612e9f33d006bb0789009aaa
Account #15: 0xcd3b766cdd6ae721141f452c550ca635964ce71 (10000 ETH)
Private Key: 0x8166f546bab6da521a8369cab06c5d2b9e46670292d85c875ee9ec20e84ffb61
Account #16: 0x2546bcd3c84621e976d8185e91a922ae77ecec30 (10000 ETH)
Private Key: 0xea6c44ac03bff858b476bba40716402b03e41b8e97e276d1baec7c37d42484a0
Account #17: 0xbda5747bfd65f08deb54cb465eb87d40e51b197e (10000 ETH)
Private Key: 0x689af8efa8c651a91ad287602527f3af2fe9f6501a/ac4b061667b5a93e037fd
Account #18: 0xdd2fd4581271e230360230f9337d5c0430bf44c0 (10000 ETH)
Private Key: 0xde9be858da4a475276426320d5e926ecf3ba460bfac56360bfa6c4c28b4ee0
Account #19: 0x8626f6940e2eb28930efb4cef49b2d1f2c9c1199 (10000 ETH)
Private Key: 0xdf57089f9ebbaccf7ba0bc227dafbffa9fc08a93fdc68e1e42411a4efcf23656e

WARNING: Funds sent on live network to accounts with publicly known private keys WILL BE LOST

eth_chainId
eth_accounts
eth_blockNumber
eth_estimateGas
eth_getBlockByNumber
eth_feeHistory
eth_maxPriorityFeePerGas
eth_sendTransaction
Contract deployment: ProofOfExploit
Contract address: 0x5FbDB2315678afecb367f032d93f642f64180aa3
Transaction: 0x300bb6403058f0d5445a8d2b6f4865c9abef3e08789e86ad223decad3799507e
From: 0xf39fdde51aadb8f64ce6ab8827279cfff9b92266
Value: 0 ETH
Gas used: 845232 of 845232
Block #1: 0x9a252099fd93146f8cfac282b9364f83d39522933186fd5ec3f4c8e15f3c84f6

eth_chainId
eth_getTransactionByHash
eth_chainId
eth_getTransactionReceipt
```

Контракт розгорнуто в блоці №1. Витрачено 845232 одиниць газу на деплой. Hardhat запущено з прив'язкою до всіх мережевих інтерфейсів для доступу з локальної мережі.

Інтерфейс реалізовано як єдиний HTML файл з підключенням Ethers.js через CDN. Браузерний провайдер отримує підписанта через MetaMask. Хешування тексту виконується локально в браузері перед відправкою транзакції.

```
<SNIP>
...
<script type="module">
const CONTRACT = "0x5FbDB2315678afecb367f032d93f642f64180aa3";
const ABI = [
  "function register(bytes32 hash, string calldata severity, string calldata target) external",
  "function verify(bytes32 hash) external view returns (address, uint256, string, string)",
  "function count() external view returns (uint256)"
];

const { ethers } = await
import("https://cdn.jsdelivr.net/npm/ethers@6.13.2/ethers.min.js");

let contract;

window.connect = async () => {
  await ethereum.request({ method: "eth_requestAccounts" });
  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer = await provider.getSigner();
  contract = new ethers.Contract(CONTRACT, ABI, signer);
}
```

```

    const addr = await signer.getAddress();
    document.getElementById("account").innerText = addr;
    const n = await contract.count();
    setStatus(`connected. total records: ${n}`, false);
};

window.register = async () => {
    const text = document.getElementById("exploitText").value;
    const severity = document.getElementById("severity").value;
    const target = document.getElementById("target").value;
    if (!text || !target) return setStatus("fill all fields", true);
    const hash = ethers.keccak256(ethers.toUtf8Bytes(text));
    setStatus("sending tx...", false);
    try {
        const tx = await contract.register(hash, severity, target);
        await tx.wait();
        setStatus(`registered. hash: ${hash}`, false);
    } catch(e) {
        setStatus(e.reason || e.message, true);
    }
};

window.verify = async () => {
    const text = document.getElementById("verifyText").value;
    if (!text) return setStatus("enter text", true);
    const hash = ethers.keccak256(ethers.toUtf8Bytes(text));
    try {
        const [researcher, timestamp, severity, target] = await
contract.verify(hash);
        const date = new Date(Number(timestamp) * 1000).toISOString();
        setStatus(`researcher: ${researcher}\ndate: ${date}\nseverity:
${severity}\ntarget: ${target}`, false);
    } catch(e) {
        setStatus("not found", true);
    }
};

function setStatus(msg, isErr) {
    const el = document.getElementById("status");
    el.innerText = msg;
    el.className = isErr ? "err" : "ok";
}
</script>
...
<SNIP>

```

// Proof of Exploit Registry

Connect

Connect MetaMask

Register Exploit

Describe the vulnerability (this stays local, only hash goes to blockchain)

CRITICAL

Target system (e.g. nginx 1.18, OpenSSL 3.0.1)

Register Hash On-Chain

Verify

Paste original vulnerability description to verify

Verify

Status

-

До MetaMask вручну додано локальну мережу з параметрами: RPC URL <http://192.168.0.165:8545>, Chain ID 31337, Імпортовано тестовий акаунт Account №0 з приватним ключем з виводу hardhat node. Початковий баланс акаунту складає 10000 тестових токенів.

Edit network

Network name

Hardhat

According to our records, the network name may not correctly match this chain ID.
Suggested name: [GoChain Testnet](#)

Default RPC URL

192.168.0.165:8545

Chain ID

31337

Currency symbol

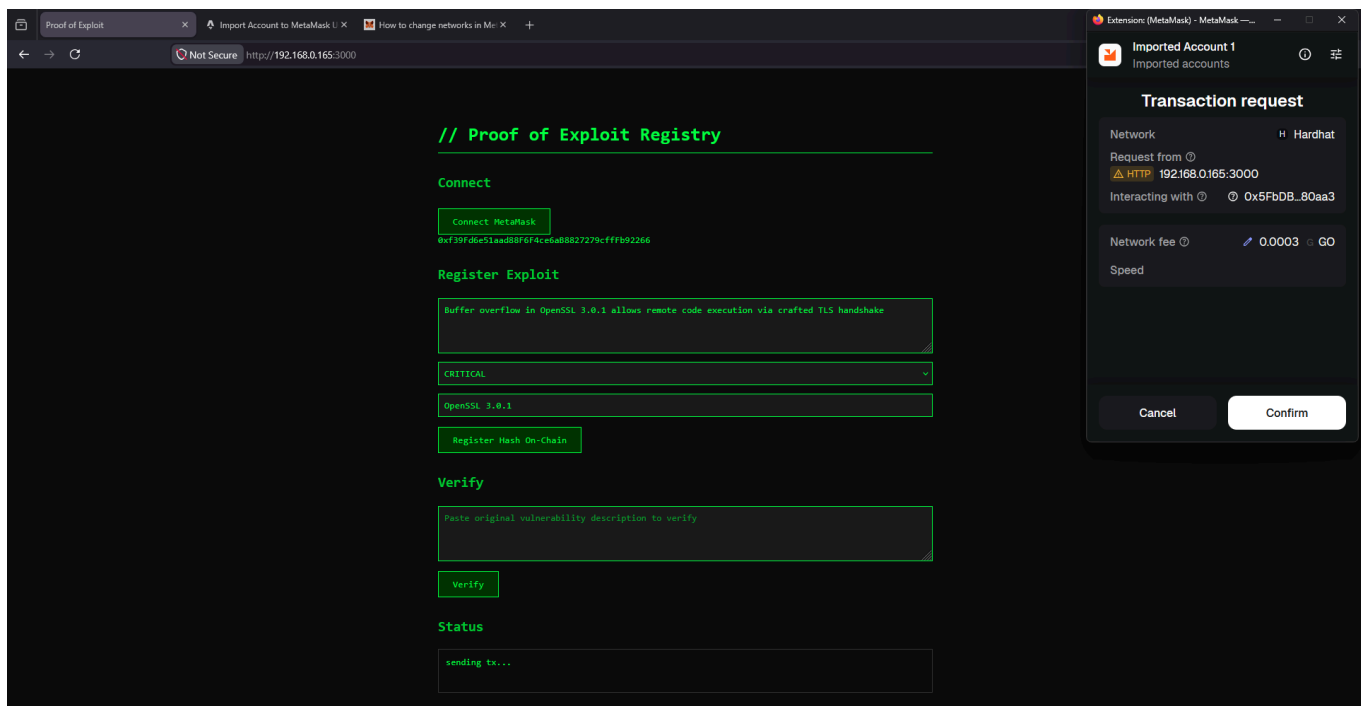
EXP

Suggested currency symbol: [GO](#)
This token symbol doesn't match the network name or chain ID entered. Many popular tokens use similar symbols, which scammers can use to trick you into sending them a more valuable token in return. Verify everything before you continue.

Block explorer URL

Add a URL

Для тестування зареєстровано вразливість з описом Buffer overflow in OpenSSL 3.0.1 allows remote code execution via crafted TLS handshake, рівень критичності CRITICAL, ціль OpenSSL 3.0.1. Браузер обчислив кессак256 від тексту і відправив хеш у транзакції до контракту. MetaMask запросив підтвердження і підписав транзакцію. Спроба повторно зареєструвати той самий хеш була відхилена контрактом з помилкою already registered - захист від дублікатів працює коректно.



second registration try

```

$ npx serve -p 3000
ERROR Cannot copy server address to clipboard: Couldn't find the 'xsel' binary and fallback
didn't work. On Debian/Ubuntu you can install xsel with: sudo apt install xsel.

Serving!
- Local: http://localhost:3000
- Network: http://192.168.0.165:3000

HTTP 5/31/2026 10:03:48 PM 192.168.0.173 GET /
HTTP 5/31/2026 10:03:48 PM 192.168.0.173 Returned 304 in 16 ms
HTTP 5/31/2026 10:15:39 PM 192.168.0.173 GET /
HTTP 5/31/2026 10:15:39 PM 192.168.0.173 Returned 304 in 2 ms
HTTP 5/31/2026 10:17:20 PM 192.168.0.173 GET /
HTTP 5/31/2026 10:17:20 PM 192.168.0.173 Returned 304 in 1 ms

```

```

Error: Transaction reverted without a reason

eth_getBalance
eth_call
Contract call: ProofOfExploit# unrecognized-selector>
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff9b92266
To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

Error: Transaction reverted without a reason

eth_call
Contract call: ProofOfExploit# unrecognized-selector>
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff9b92266
To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

Error: Transaction reverted without a reason

eth_call
Contract call: ProofOfExploit# unrecognized-selector>
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff9b92266
To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

Error: Transaction reverted without a reason

eth_blockNumber
eth_gasPrice
eth_blockNumber
eth_getBlockByNumber
eth_blockNumber
eth_call
Contract call: ProofOfExploit#count
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff9b92266
To: 0x5fbdb2315678afecb367f032d93f642f64180aa3

eth_blockNumber (2)
eth_estimateGas
Contract call: ProofOfExploit#register
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfff9b92266
To: 0x5fbdb2315678afecb367f032d93f642f64180aa3
Value: 0 ETH

Error: reverted with reason string 'already registered'

eth_blockNumber (4)

```

Verify

```
Buffer overflow in OpenSSL 3.0.1 allows remote code execution via crafted TLS handshake
```

Verify

Status

```
researcher: 0xf39Fd6e51aad88F6F4ce6aB8827279cFfb92266  
date: 2026-05-31T19:15:11.000Z  
severity: critical  
target: OpenSSL 3.0.1
```

Після введення оригінального тексту вразливості контракт повернув повну інформацію про запис: адреса дослідника 0xf39Fd6e51aad88F6F4ce6aB8827279cFfb92266, дата реєстрації 2026-05-31T19:15:11.000Z, рівень критичності critical, ціль OpenSSL 3.0.1. Верифікація підтвердила що хеш зберігається у блокчейні і авторство зафіксовано незмінно.

Децентралізована архітектура усуває єдину точку відмови - дані зберігаються у блокчейні і не залежать від центрального сервера. Незмінність записів гарантується консенсусом мережі: після включення транзакції до блоку запис не може бути видалений або змінений. Конфіденційність деталей вразливості забезпечується тим що у блокчейн записується лише хеш, а не сам текст. Захист від підробки авторства забезпечується криптографічним підписом транзакції приватним ключем дослідника.

У ході лабораторної роботи розроблено децентралізований додаток Proof of Exploit на платформі Ethereum з використанням Hardhat і Solidity. Додаток реалізує захист інтелектуальної власності дослідників безпеки через фіксацію хешу опису вразливості у блокчейні з прив'язкою до адреси автора і часу. Перевірено коректну роботу реєстрації, захист від дублікатів і верифікацію авторства через браузерний інтерфейс з MetaMask.